

AgentDSS: A Human-in-the-Loop Sequential Decision Pipeline for Supply Chain Disruption Management

Dontha Meghashyam Sai, H Yashwanth, Dvns Yashwanth

Abstract—Supply chain disruptions are typically managed through siloed, department-level decisions that ignore downstream consequences and give human managers no structured way to inject real-world knowledge that automated systems lack. This proposal presents AgentDSS, a human-in-the-loop sequential decision pipeline in which four role-specific large language model (LLM) agents (Supplier, Inventory, Logistics, Finance) generate decision options conditioned on all prior human decisions in the chain. At every step, a human may accept an agent’s option or supply an unconstrained override, which is propagated as context to every subsequent agent. A deterministic Impact Preview heuristic estimates downstream consequences before a decision is confirmed. Unlike prior work in this space, this proposal treats evaluation as a first-class deliverable: we specify a synthetic scenario suite generated under a stratified sampling scheme, a controlled comparison between override-enabled and no-override baseline conditions scored by an independent judge model, a pre-registered statistical test with correction for multiple comparisons, a small-scale pilot validation phase preceding full-scale evaluation, and explicit reproducibility and reporting standards designed so results can be independently verified rather than only plausibly demonstrated.

Index Terms—decision support systems, large language model agents, human-in-the-loop systems, supply chain management, evaluation methodology, reproducibility

I. INTRODUCTION

Modern supply chains are managed through department-level systems — supplier, inventory, logistics, and finance — that operate largely in isolation. When a disruption occurs, three structural problems recur. First, siloed decision-making: departments react independently, sometimes producing directly contradictory actions. Second, the absence of a human override mechanism: existing automated decision support systems (DSS) generate recommendations algorithmically with no structured channel for a manager’s real-world knowledge to alter downstream computation. Third, blind decision-making: a manager acting at one point in the chain has no visibility into the consequences their decision will trigger further downstream.

This proposal describes AgentDSS, a system in which each department is represented by a role-specific LLM agent operating over its own dataset. Agents generate options sequentially, each conditioned on the full history of prior human decisions, so that an override at any step changes the options generated by every subsequent agent. This is deliberately framed as a human-in-the-loop sequential decision pipeline rather than a multi-agent system in the negotiation-based sense: agents do

not communicate with, or reach consensus with, one another. All coordination passes through a single shared decision log.

II. PROBLEM STATEMENT

A. Siloed Decision-Making

Departmental systems act on local information only, with no shared record of decisions made elsewhere in the chain during a disruption event.

B. Absence of Human Override

A manager who knows that stock can be sourced informally, or that a supplier relationship has off-the-record flexibility, has no way to inject that knowledge into an automated recommendation pipeline.

C. Blind Downstream Consequences

A decision made at one stage of the chain is committed without visibility into its effect on later stages, financially or operationally.

III. PROPOSED SOLUTION

AgentDSS addresses each problem directly: shared context eliminates siloing, since every agent conditions on the complete decision history; free-text override at every step eliminates the absence of human input; and a deterministic Impact Preview heuristic (Section V-A) eliminates blind decision-making by estimating downstream consequences before a decision is confirmed.

TABLE I
PROBLEM-TO-MECHANISM MAPPING

Problem	Mechanism
Siloed decisions	Every agent receives the complete prior decision history
No human override	Free-text override accepted at every step; propagated immediately
Blind decision-making	Deterministic Impact Preview surfaces estimated downstream effects pre-confirmation

IV. SYSTEM OPERATION

The pipeline executes as follows: (1) the team configures up to four agents, each assigned a role and a dataset partition; (2) the operator enters a disruption description; (3) Agent 1 reads its dataset and the disruption description and returns two to three ranked options with a confidence score; (4) the human accepts an option or supplies a free-text override; (5) the Impact Preview Engine computes an estimated downstream effect using a precomputed heuristic formula rather than live agent calls; (6) the confirmed decision is written to the decision log and Context Manager; (7) Agent 2 reads the updated context and generates new options reflecting the current state; (8) steps 4–7 repeat through Agent 4; (9) a complete decision log is produced.

V. SYSTEM ARCHITECTURE

The backend consists of four components: the **Agent Engine**, which reads dataset and context and calls the Claude API to return ranked options with confidence; the **Context Manager**, which maintains the decision log and stores overrides against an explicit decision-event schema (Section VI); the **Impact Preview Engine**, which estimates downstream consequence using the heuristic defined in Section V-A rather than live parallel LLM calls to remaining agents; and the **Decision Log Engine**, which records every recommendation, confidence score, human decision, and rationale.

A. Impact Preview Heuristic

The Impact Preview score for a candidate decision d is computed as a weighted sum over three normalized deltas drawn directly from the backbone dataset:

$$\text{Impact}(d) = w_c \cdot \Delta\text{cost}(d) + w_t \cdot \Delta\text{leadtime}(d) + w_r \cdot \Delta\text{risk}(d) \quad (1)$$

where Δcost , $\Delta\text{leadtime}$, and Δrisk are min-max normalized to $[0, 1]$ relative to the scenario’s baseline state, and weights w_c, w_t, w_r (summing to 1) are fixed prior to evaluation and reported alongside results. This formulation is deterministic given the dataset and decision, and is reproducible across repeated scenario runs, whereas a live-LLM preview introduces run-to-run variance that would confound comparison between the override and baseline conditions in Section VII.

VI. DECISION EVENT SCHEMA

Each decision event is stored as a structured record rather than free text, so override behavior can be queried and scored programmatically during evaluation: `event_id`, `agent_role`, `options_presented` (with confidence scores), `decision_type` (accepted_option | override), `decision_text`, `override_flag`, `preview_shown`, and `timestamp`.

VII. EVALUATION METHODOLOGY

This section is the central technical contribution of the project and is treated as a first-class deliverable rather than a final-phase validation step.

A. Pilot Validation

Before full-scale evaluation, the complete pipeline is run end-to-end on a pilot set of 10 scenarios to confirm that judge scores, heuristic outputs, and override propagation all behave as expected. Full-scale scenario generation and evaluation proceed only after the pilot set passes a manual sanity check; this ordering is a binding process constraint, not an optional step.

B. Scenario Suite

Sixty synthetic disruption scenarios are generated from a real-world backbone dataset (a Southern California logistics network dataset providing supplier reliability scores, lead times, route risk levels, and disruption-likelihood indicators), stratified evenly across four disruption templates (supplier delay, demand spike, logistics route failure, combined multi-factor disruption) at 15 scenarios per template. Within each template, scenarios are generated by varying disruption parameters (severity, affected node, time horizon) across the full range supported by the backbone dataset, so that the 15 scenarios per template are not near-duplicates of one another; parameter ranges and the sampling scheme are reported in the final report to allow independent verification of scenario diversity.

C. Experimental Conditions

Each scenario is run under two conditions: (1) a no-override baseline, in which each agent’s top-confidence option is accepted automatically; and (2) an override condition, in which a scripted override policy injects at least one deviation per scenario. Scripted overrides keep scenario runs deterministic and repeatable; a small number of live human-run sessions are additionally recorded for qualitative comparison and are reported separately from the scripted-run statistics.

D. Scoring

Each completed run is scored on: (1) a decision-quality rubric evaluated by an LLM-as-judge configuration using a model from a different provider family than the agent-generation model, to avoid same-family evaluation bias; agreement between the judge model and a human-rated subsample is reported as a Cohen’s kappa or Spearman correlation, not only as a qualitative spot-check; (2) cost and logistics deltas computed directly from the backbone dataset’s fields; and (3) an override-acceptance pattern analysis.

E. Ablation Studies

- Does the Impact Preview measurably change the decision that follows it, compared to a run where the preview is suppressed?
- Does agent confidence score correlate with observed override rate?
- Does dynamic, context-conditioned option generation produce options that differ meaningfully from a static baseline in which each agent ignores prior decisions?

F. Statistical Treatment

The primary override-vs-baseline comparison and each ablation are evaluated with the Wilcoxon signed-rank test on paired scenario runs, selected over a paired t -test because rubric scores are ordinal and not assumed normally distributed. Across the four planned comparisons (main comparison plus three ablations), a Bonferroni correction is applied to the significance threshold. All test statistics, exact p -values, and effect sizes are reported in full, not reduced to a binary significant/not-significant summary.

VIII. REPRODUCIBILITY AND REPORTING STANDARDS

This project commits to standards commonly absent from comparable LLM-agent evaluations, treated as binding constraints on the eval harness rather than aspirational goals.

A. Reproducibility

- Scenario generation uses a fixed random seed; the same seed reproduces the same 60 scenarios on any run.
- Agent API calls during evaluation runs use a fixed, low temperature so repeated runs are comparable.
- Every prompt version used is version-controlled and timestamped; prompts are never silently edited without a recorded revision, and any prompt revision after the pilot phase triggers a re-run of affected scenarios.
- Raw output of every scenario run is stored, not only the final aggregate score.

B. Reporting

- Override-vs-baseline results are reported as exact counts, test statistics, and p -values, not rounded to qualitative claims.
- Any ablation result that does not support a hypothesis is published alongside results that do, rather than omitted.
- API cost and end-to-end latency per scenario run are disclosed as part of the final report.

IX. TECHNOLOGY STACK

Backend: Python, FastAPI. LLM engine: Claude API (agent generation), separate-provider model (judge scoring). Dataset processing: Pandas. Database: PostgreSQL. Frontend: React (separate track).

X. RESEARCH GAP AND NOVELTY

Existing work in this space includes agentic consensus-seeking among LLM agents in inventory management [1], standalone zero-shot inventory agents without inter-agent communication [2], retailer-specific planning agents built for a single company [4], and multi-agent risk assessment scoped narrowly to logistics hub selection [5]. None of the surveyed systems offers user-configurable agents, an unconstrained human override channel, a downstream consequence preview prior to commitment, or — critically — a published comparative evaluation of override-enabled operation against a no-override baseline, scored by an independent judge and reported with pre-registered statistical treatment and reproducibility guarantees.

XI. PROJECT PHASES

This project is structured as phase-based milestones with explicit track ownership across the three-person team: Track A — Agent Engine and Prompting; Track B — Context Manager, Impact Preview, Decision Log, and Eval Harness; Track C — API, Frontend, and Integration. Eval-harness and reproducibility milestones are embedded within Track B rather than deferred to a final validation week. Six phases: Phase 1 — API contract definition, Context Manager schema, Agent 1 minimal version (Tracks A/B/C, parallel); Phase 2 — Agent 2 connected, context-dependent option generation validated as the highest-risk assumption (Track A); Phase 3 — Impact Preview heuristic implementation and scenario-suite generation running in parallel, not sequentially (Track B); Phase 4 — Agents 3–4 connected, decision record schema finalized, pilot validation run on 10 scenarios, baseline-vs-override test runner built (Track B); Phase 5 — frontend build and full end-to-end wiring (Track C); Phase 6 — full evaluation run across the 60-scenario suite, ablations, statistical analysis, reproducibility verification, public deployment, and write-up (all tracks).

XII. EXPECTED OUTCOMES

- A deployed, working pipeline demonstrating the full agent cycle with human override at every step.
- A published comparative evaluation across the full 60-scenario suite, with pre-registered significance testing and effect sizes.
- Ablation results addressing whether each core mechanism produces a measurable effect.
- A complete, exportable decision log per scenario run.
- A written technical report meeting the reproducibility and reporting standards in Section VII, suitable for public portfolio use.

REFERENCES

- [1] V. Jannelli et al., “Agentic LLMs in the supply chain: Towards autonomous multi-agent consensus-seeking,” arXiv:2411.10184, 2024.
- [2] Y. Quan and Z. Liu, “InvAgent: A large language model based multi-agent system for inventory management in supply chains,” arXiv:2407.11384, 2024.
- [3] T. Guo et al., “Large language model based multi-agents: A survey of progress and challenges,” arXiv:2402.01680, 2024.
- [4] “Leveraging LLM-based agents for intelligent supply chain planning,” JD.com case study, arXiv:2509.03811, 2025.
- [5] M. Zhao et al., “Leveraging large language models to enhance multi-agent risk assessment in supply chain networks (MARS framework),” Int. J. Prod. Res., 2026.
- [6] D. J. Power, *Decision Support Systems: Concepts and Resources for Managers*. Westport, CT: Quorum Books, 2002.